

Arquitetura de Computadores

Exercício Prático 01

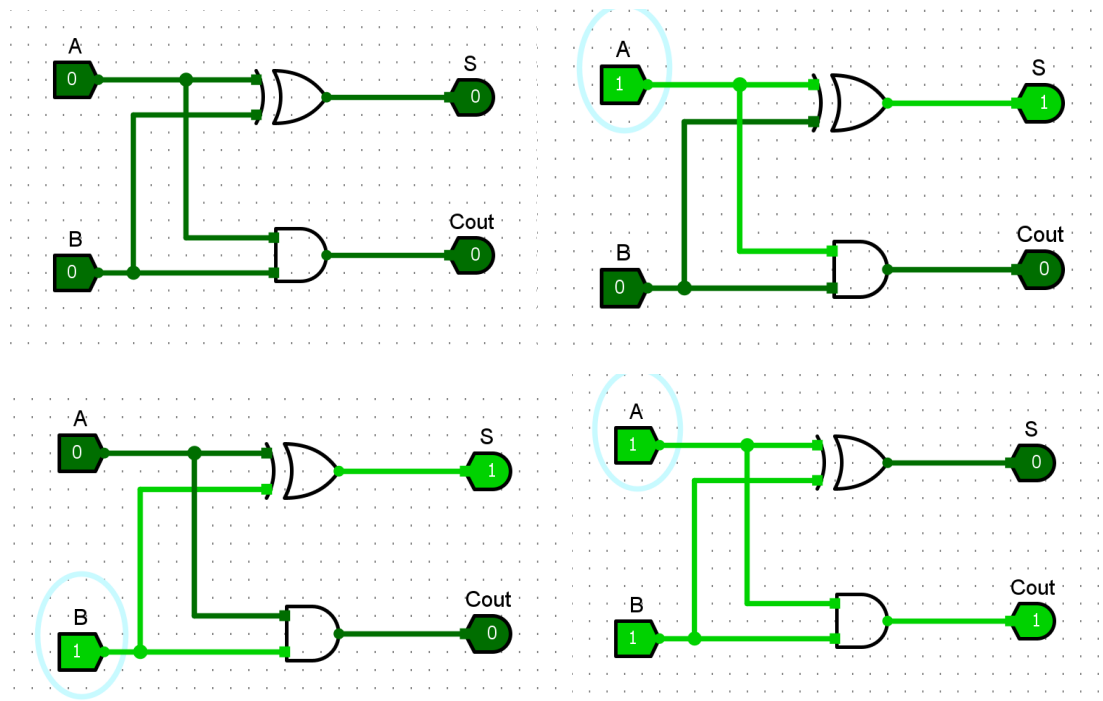
Aluno: Miro Brito de Arêa Leão

Matrícula: 892184

Parte 1:

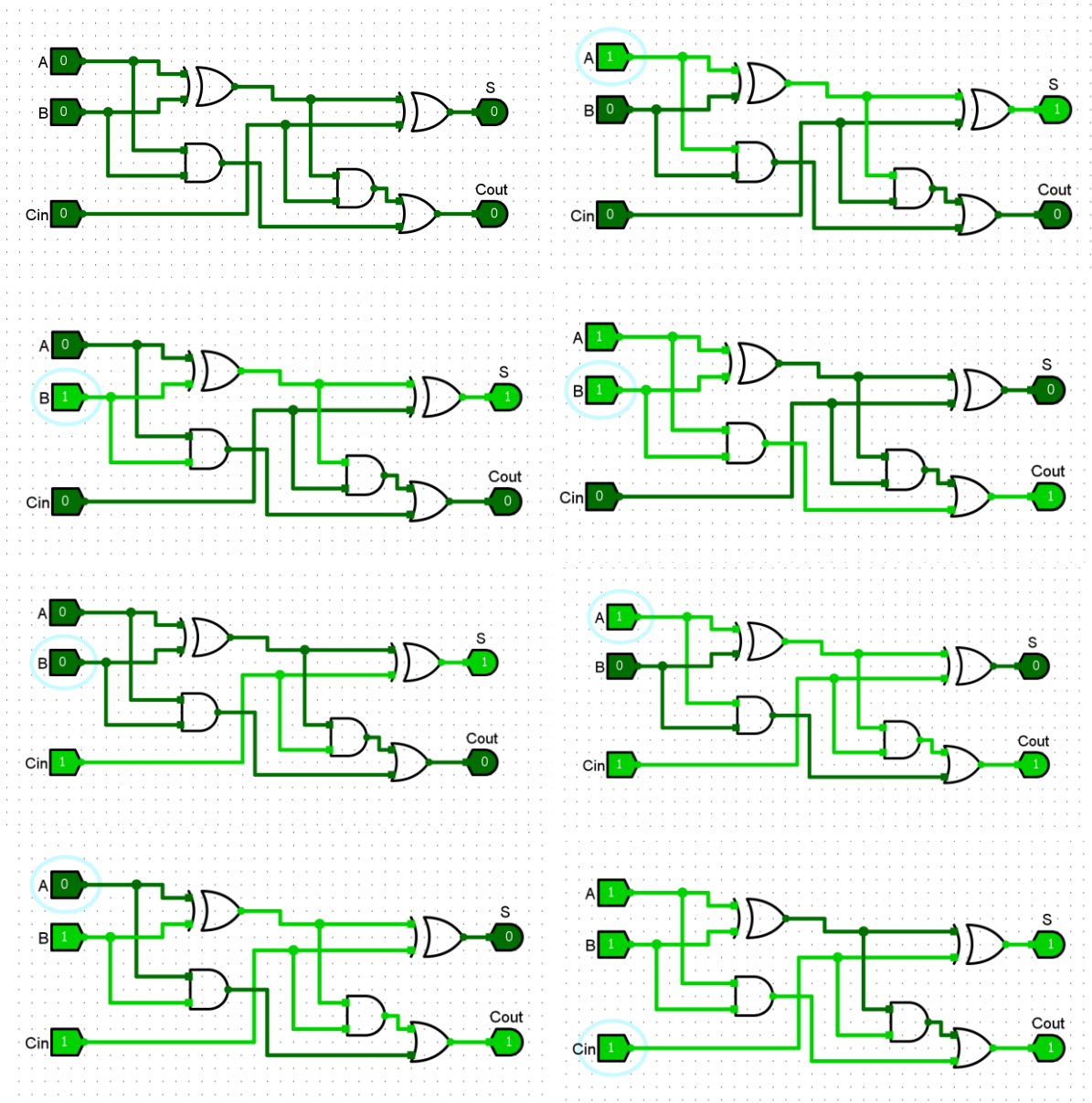
1) Imagens do circuito de um $\frac{1}{2}$ somador:

As imagens mostram todos os testes feitos nesse somador.



2) Imagens do circuito de um somador completo:

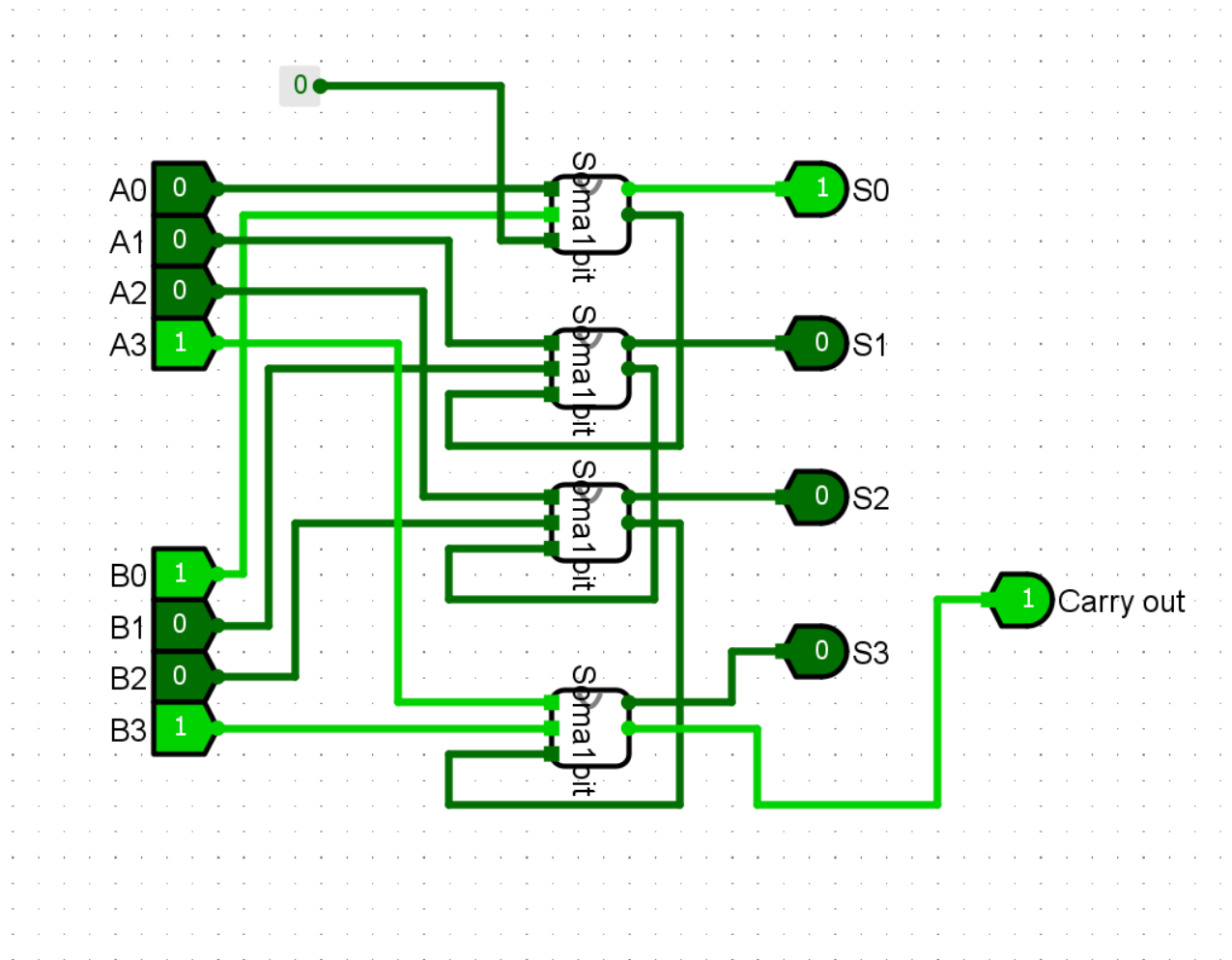
As imagens mostram todos os testes feitos nesse somador.



3) Imagem do circuito de um somador de 4 bits

Números utilizados na soma: 8 e 9

Mat: 892184 - Aluno: Miro Leão



4) Perguntas respondidas:

1) O principal problema de tempo desse somador é o atraso na propagação do carry, pois nesse somador o vai-um (carry) precisa se propagar do bit menos significativo (bit 0) até o mais significativo (bit 3), acumulando o tempo decorrente de cada etapa, já que cada estágio precisa aguardar o resultado do estágio anterior. Isso gera um atraso total que cresce com o número de bits.

2) Considerando atraso médio de 10 ns por porta lógica e aproximadamente 3 portas no caminho crítico, o tempo por estágio é cerca de 30 ns.

Para 4 bits o atraso é de 90ns, pois ele passa por 4 estágios, onde no primeiro gasta 30ns (somador), porém nos outros 3 ele gasta somente 20ns, pois o processo ocorre em paralelo, somando no final 90ns.

3) Seria necessário encadear 32 subcircuitos de somadores completos de 1 bit. O carry de saída de cada bit deve ser conectado obrigatoriamente ao carry de entrada do bit subsequente ($Co(i) \rightarrow Ci(i+1)$), criando uma longa corrente de dependência.

4) Para 32 bits:

$$32 \times 30 \text{ ns} = 960 \text{ ns.}$$

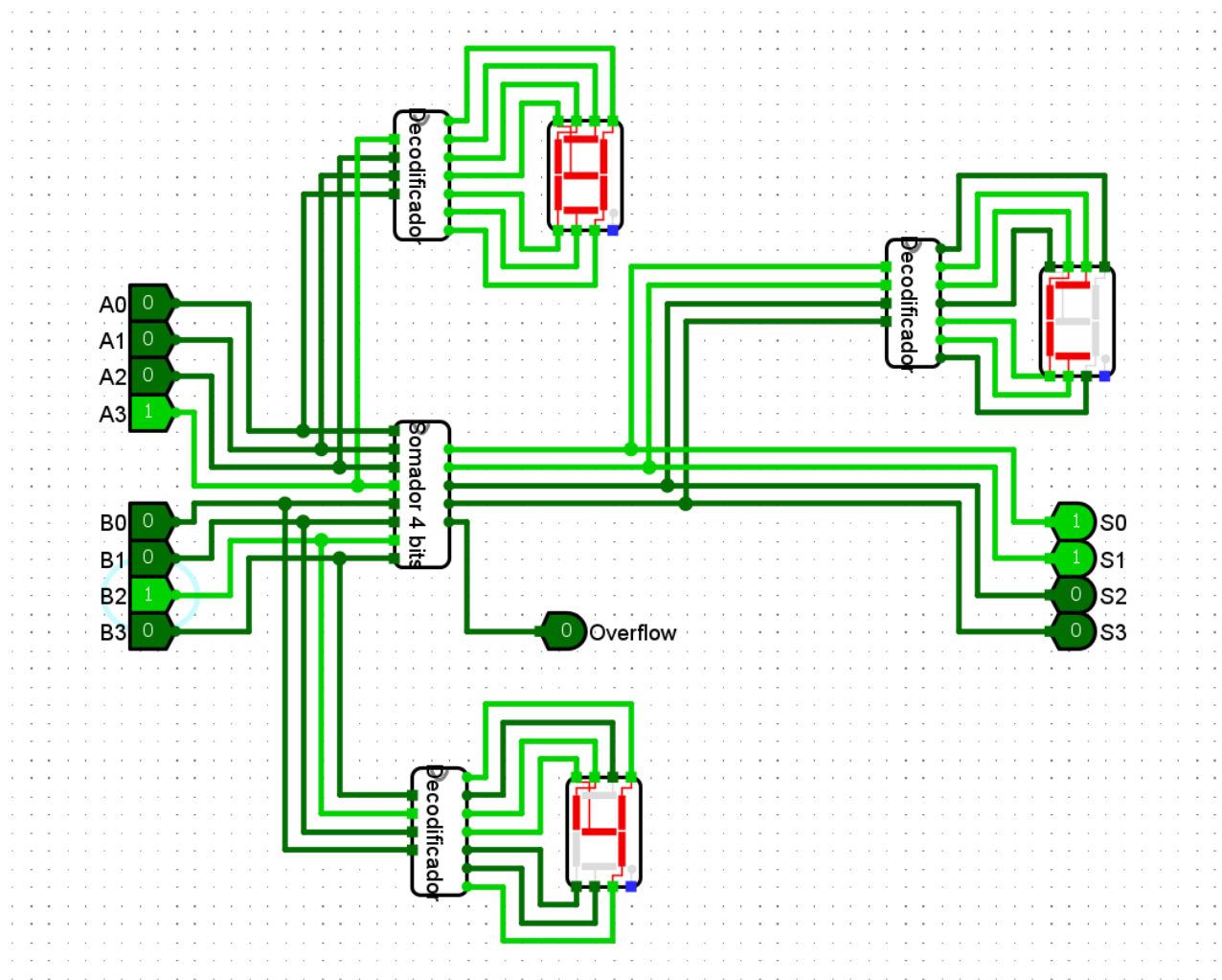
$$\text{Frequência aproximada: } 1 / 960 \text{ ns} \approx 1,04 \text{ MHz.}$$

5) Uma forma de tornar a soma mais veloz é utilizar arquiteturas como Carry Lookahead Adder.

Parte 2:

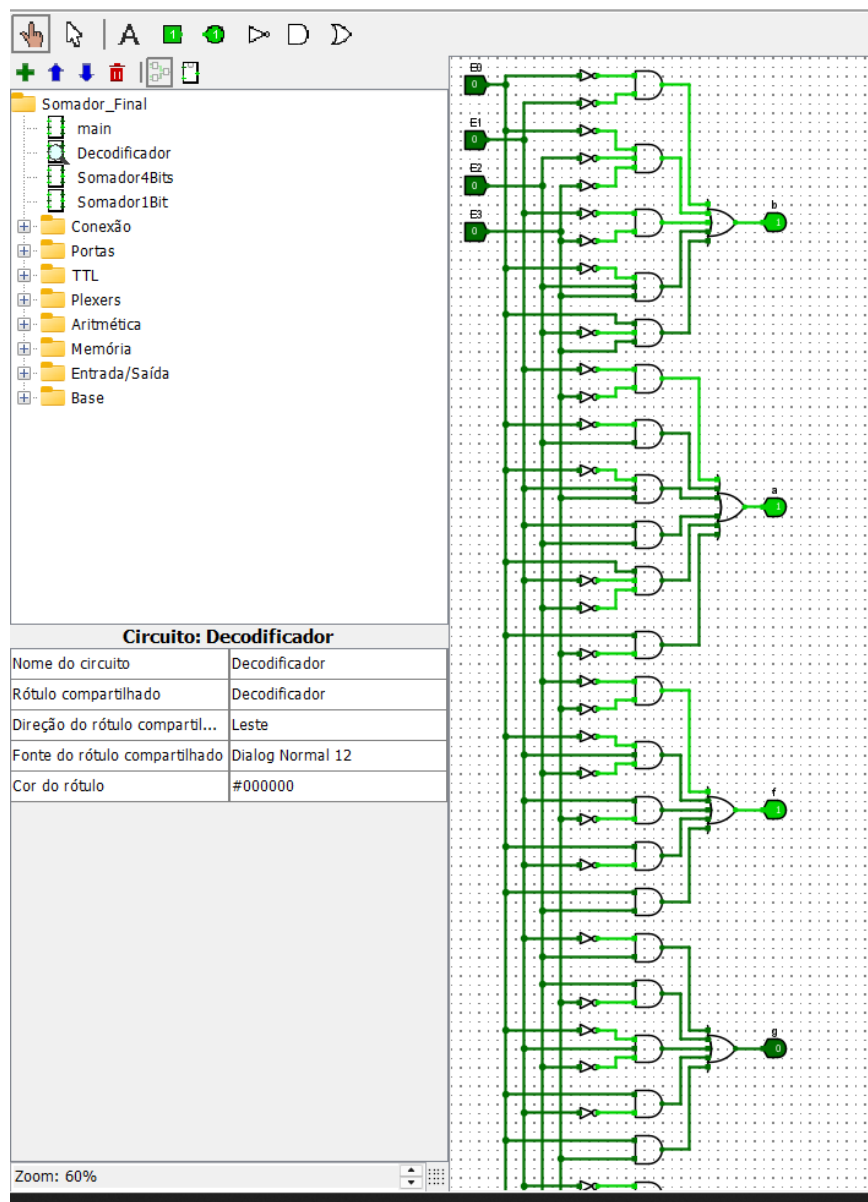
- 1) Imagem do circuito de um somador de 4 bits
Números utilizados na soma: 8 e 4

Mat: 892184 - Aluno: Miro Leão

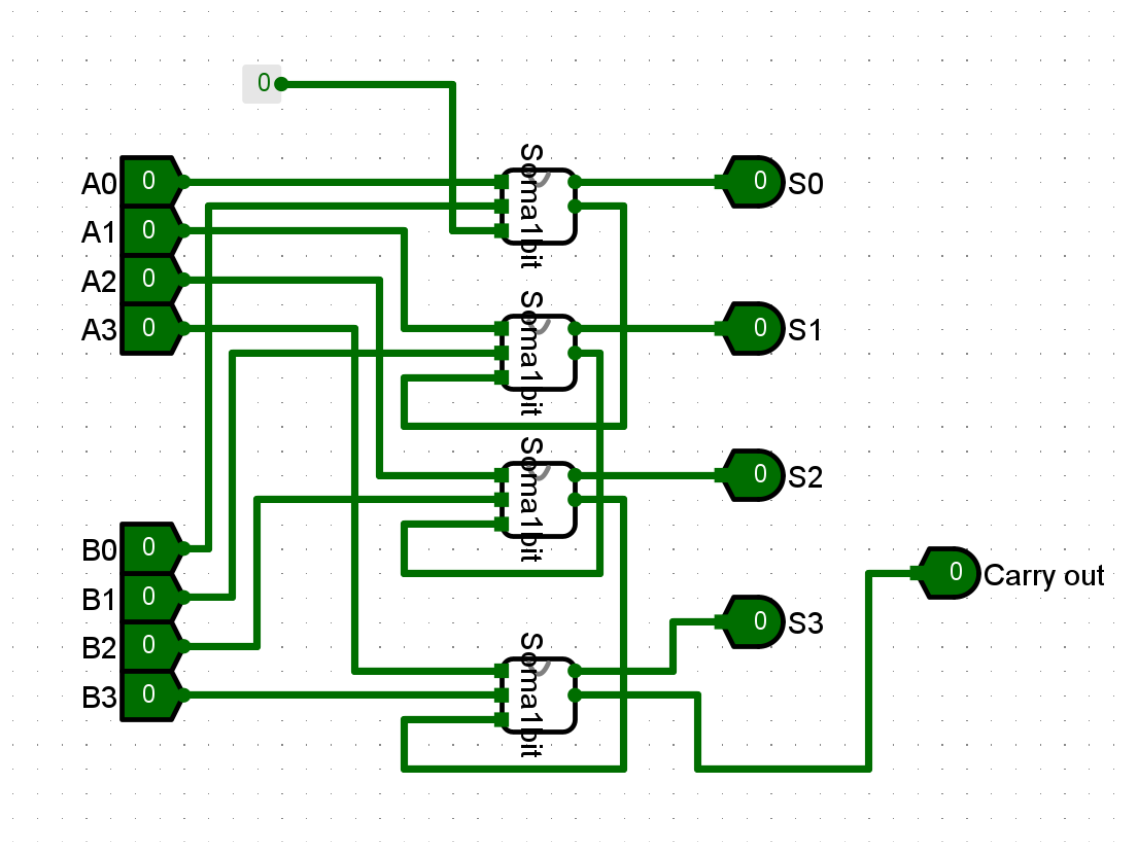


2) Imagens das partes internas e subcircuitos:

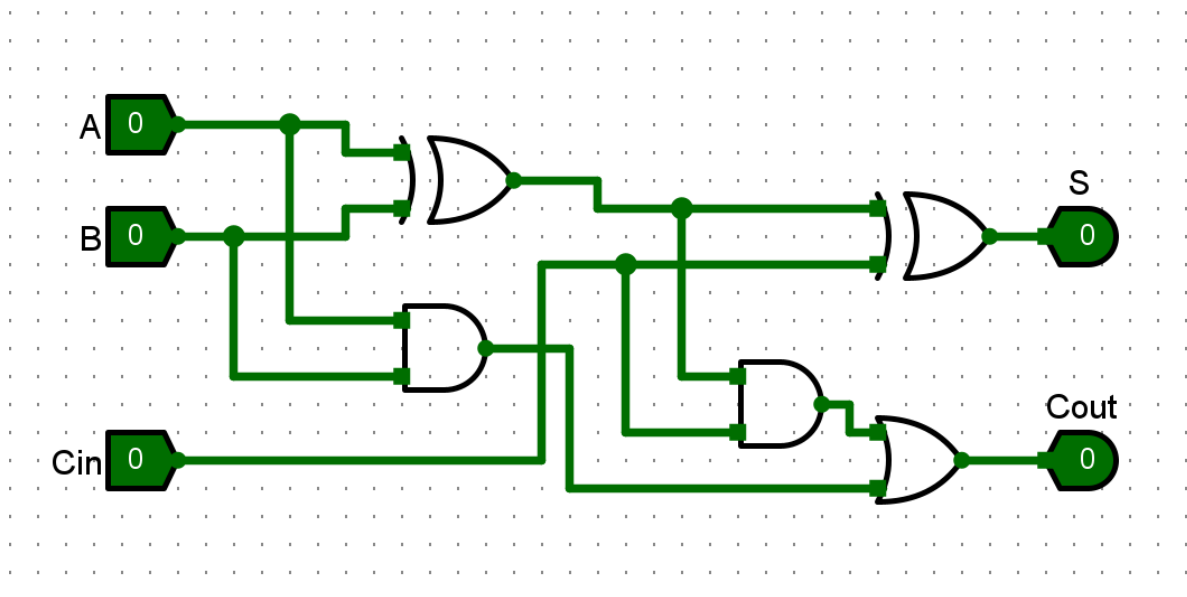
- Decodificador:



- Somador 4 bits:



- Somador 1 bit:



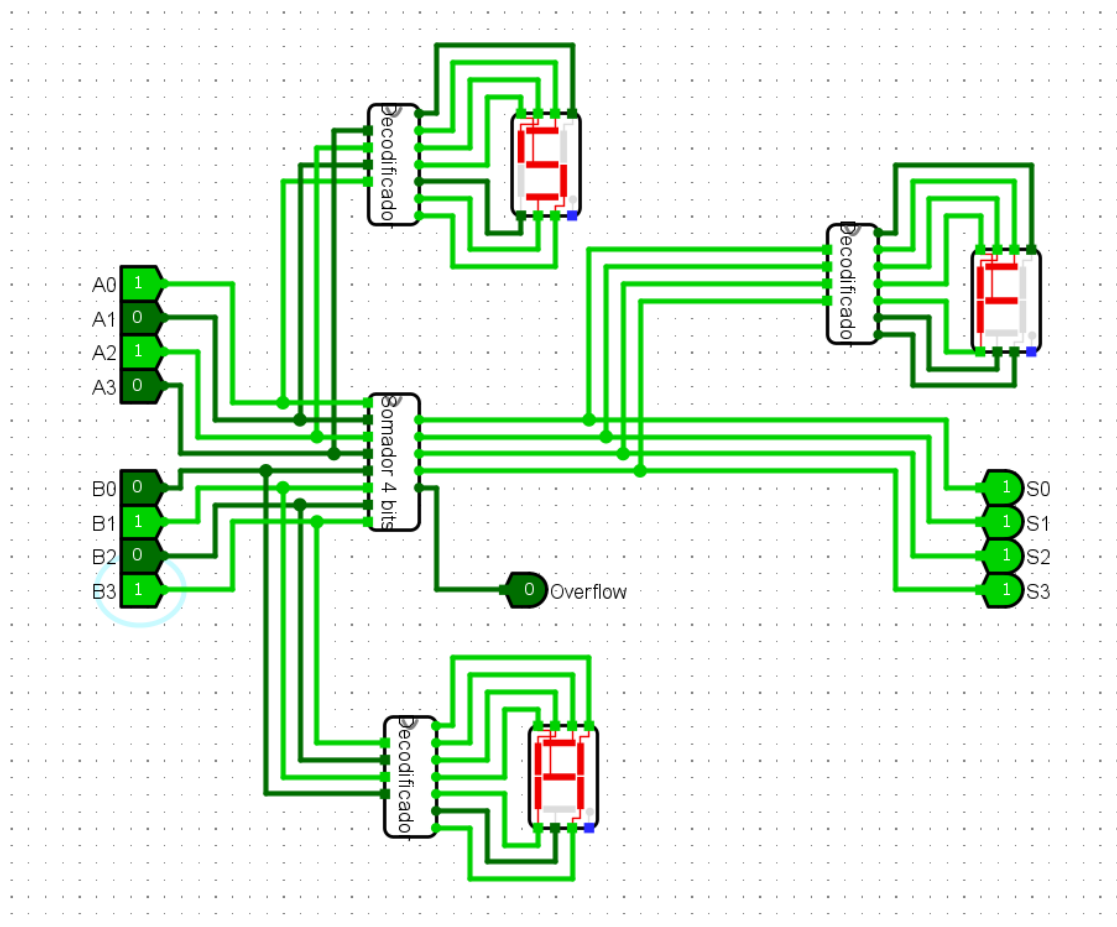
Parte 3:

a) +5 + (-6):

- Binário: 0101 (+5) + 1010 (-6) = 1111 (-1)

- **Display Entrada:** A -> 5 (0101) e B -> A (1010).

- **Display Saída:** Mostrará F (representa o -1 em complemento de 2).

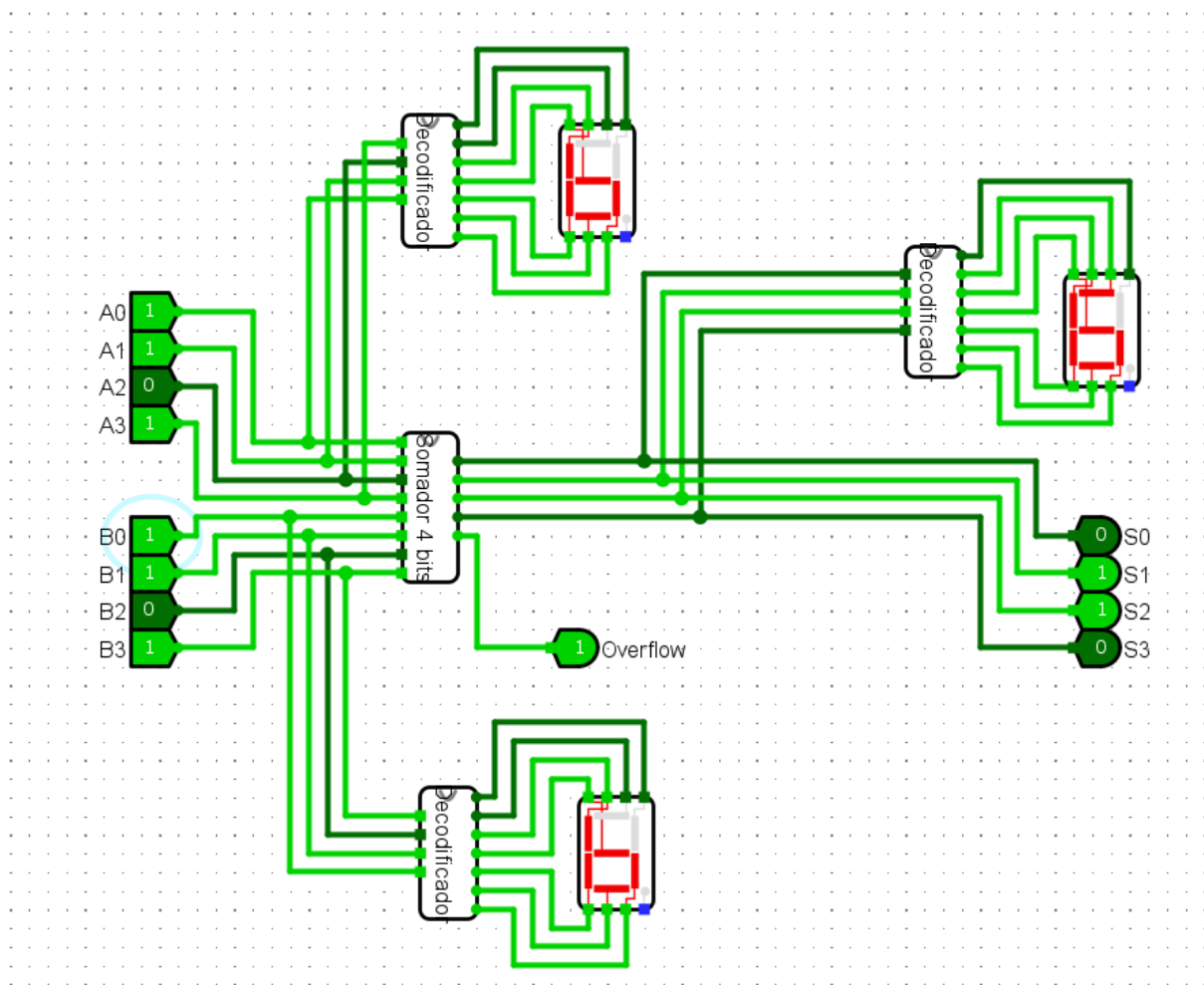


b) $-5 + (-5) = -10$

- Binário: 1011 (-5) + 1011 (-5) = 10110 . Em 4 bits, mantemos apenas 0110 .

- **Display Entrada:** A $\rightarrow$ B (1011) e B $\rightarrow$ B (1011).

- **Display:** Mostrará **6**. (Obs: este é um resultado incorreto matematicamente causado por overflow, pois -10 não cabe em 4 bits).

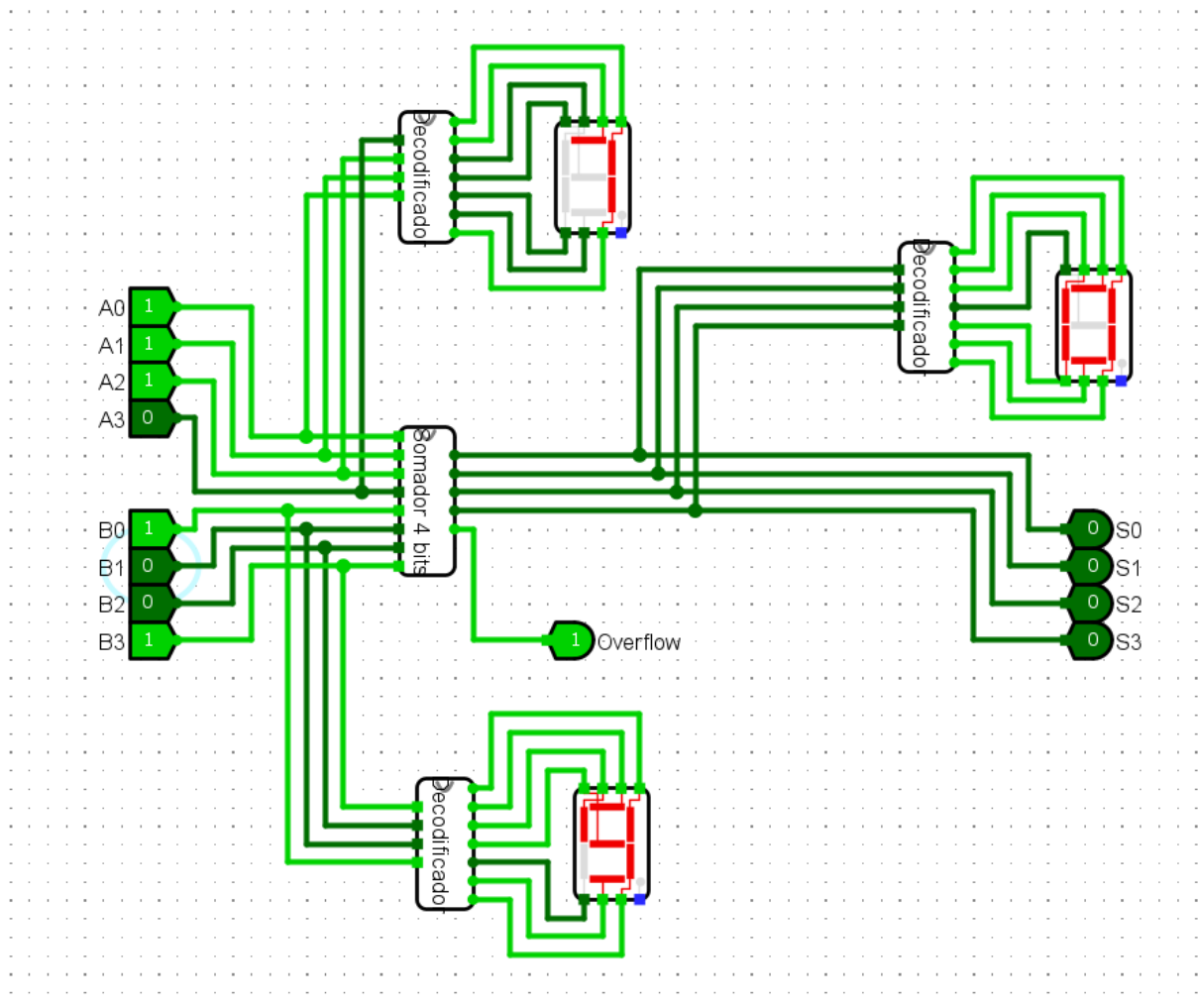


c) $-7 + (-9) = -16$

- O número -9 não pode ser representado em 4 bits (exigiria 5 bits: 10111).

d) $+7 + 9 = +16$

- O número +9 também não pode ser representado como positivo em 4 bits (o binário 1001 representa -7 em complemento de 2).



Demonstração visual do estouro de limite na operação D ($+7 + 9$). O circuito interpreta a entrada 1001 (9) como -7 em complemento de 2, resultando na soma $+7 + (-7) = 0$, ativando a flag de erro/overflow.

Obs.: Com 4 bits, podemos representar somente: **-8 até +7**.

Respostas para as Perguntas da Parte 3:

1) Porque as duas últimas operações irão ligar o overflow?

Porque os resultados matemáticos dessas somas (-16 e +16) ultrapassam completamente a capacidade de representação do somador de 4 bits em complemento de 2, cujo limite é de **-8 a +7**. O *overflow* ocorre sempre que a soma de dois números com o mesmo sinal gera um resultado que não cabe no limite de bits, o que faz com que o bit de sinal do resultado seja invertido de forma errada.

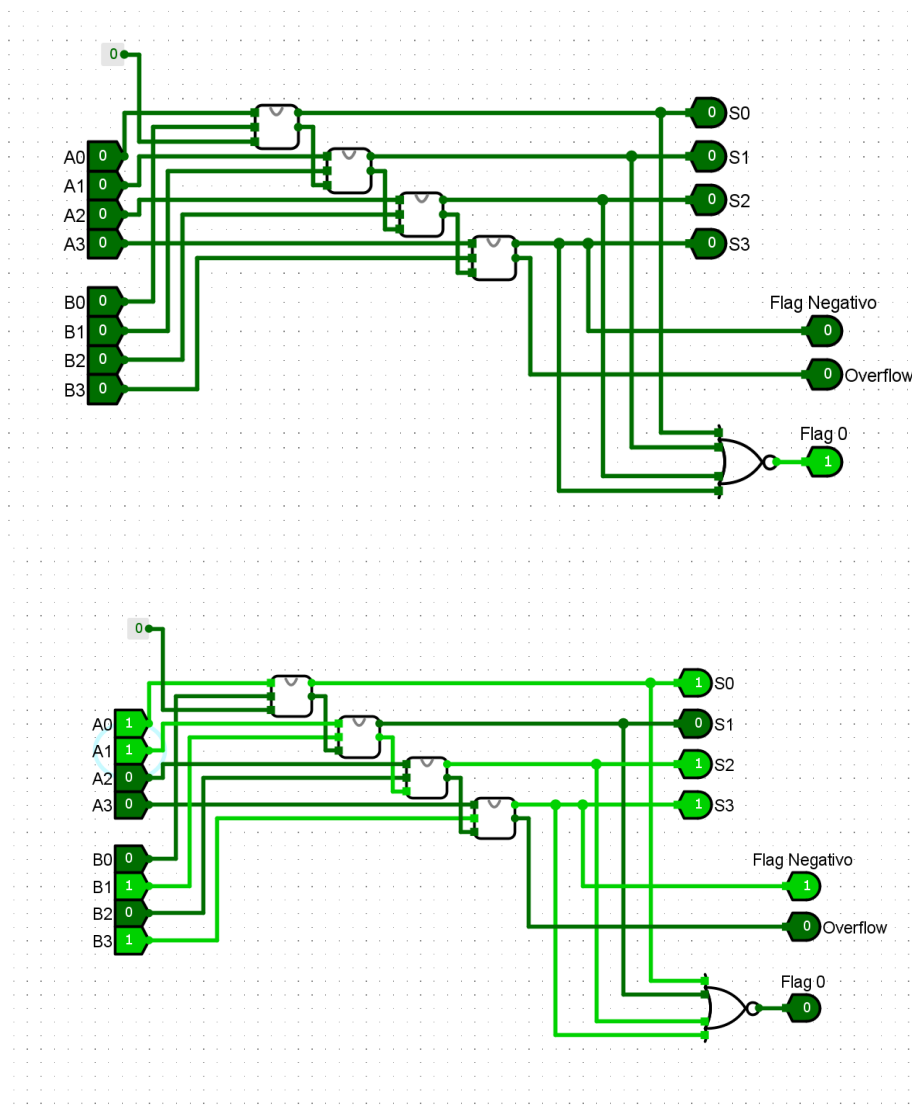
2) Qual a porta lógica que você utilizou para esta operação de identificar o overflow?

Foi utilizada a porta **XOR**. O circuito identifica o *overflow* realizando a operação lógica XOR entre o *carry in* que entra no bit mais significativo (bit 3) e o *carry out* que sai do bit mais significativo. Se eles forem diferentes, a porta XOR emite nível lógico alto (1), ativando a flag.

Parte 4:

1) Imagens dos circuitos com os flags inseridos

- Subcircuito (somador 4 bits):



- Somador completo com as flags:

